

Using uv with Jupyter

The Jupyter notebook is a popular tool for interactive computing, data analysis, and visualization. You can use Jupyter with uv in a few different ways, either to interact with a project, or as a standalone tool.

Using Jupyter within a project

If you're working within a project, you can start a Jupyter server with access to the project's virtual environment via the following:

```
$ uv run --with jupyter jupyter lab
```

By default, `jupyter lab` will start the server at `http://localhost:8888/lab`.

Within a notebook, you can import your project's modules as you would in any other file in the project. For example, if your project depends on `requests`, `import requests` will import `requests` from the project's virtual environment.

If you're looking for read-only access to the project's virtual environment, then there's nothing more to it. However, if you need to install additional packages from within the notebook, there are a few extra details to consider.

Creating a kernel

If you need to install packages from within the notebook, we recommend creating a dedicated kernel for your project. Kernels enable the Jupyter server to run in one environment, with individual notebooks running in their own, separate environments.

In the context of uv, we can create a kernel for a project while installing Jupyter itself in an isolated environment, as in `uv run --with jupyter jupyter lab`. Creating a kernel for the project ensures that the notebook is hooked up to the correct environment, and that any packages installed from within the notebook are installed into the project's virtual environment.

To create a kernel, you'll need to install `ipykernel` as a development dependency:

```
$ uv add --dev ipykernel
```

Then, you can create the kernel for project with:

```
$ uv run ipython kernel install --user --env VIRTUAL_ENV $(pwd)/.venv --name=project
```

From there, start the server with:

```
$ uv run --with jupyter jupyter lab
```

When creating a notebook, select the project kernel from the dropdown. Then use `!uv add pydantic` to add `pydantic` to the project's dependencies, or `!uv pip install pydantic` to install `pydantic` into the project's virtual environment without persisting the change to the project `pyproject.toml` or `uv.lock` files. Either command will make `import pydantic` work within the notebook.

Installing packages without a kernel

If you don't want to create a kernel, you can still install packages from within the notebook. However, there are a few caveats to consider.

Though `uv run --with jupyter` runs in an isolated environment, within the notebook itself, `!uv add` and related commands will modify the *project's* environment, even without a kernel.

For example, running `!uv add pydantic` from within a notebook will add `pydantic` to the project's dependencies and virtual environment, such that `import pydantic` will work immediately, without further configuration or a server restart.

However, since the Jupyter server is the "active" environment, `!uv pip install` will install package's into *Jupyter's* environment, not the project environment. Such dependencies will persist for the lifetime of the Jupyter server, but may disappear on subsequent jupyter invocations.

If you're working with a notebook that relies on `pip` (e.g., via the `%pip` magic), you can include `pip` in your project's virtual environment by running `uv venv --seed` prior to starting the Jupyter server. For example, given:

```
$ uv venv --seed
$ uv run --with jupyter jupyter lab
```

Subsequent `%pip install` invocations within the notebook will install packages into the project's virtual environment. However, such modifications will *not* be reflected in the project's `pyproject.toml` or `uv.lock` files.

Using Jupyter as a standalone tool

If you ever need ad hoc access to a notebook (i.e., to run a Python snippet interactively), you can start a Jupyter server at any time with `uv tool run jupyter lab`. This will run a Jupyter server in an isolated environment.

Using Jupyter with a non-project environment

If you need to run Jupyter in a virtual environment that isn't associated with a project (e.g., has no `pyproject.toml` or `uv.lock`), you can do so by adding Jupyter to the environment directly. For example:

=== "macOS and Linux"

```
```console
$ uv venv --seed
$ uv pip install pydantic
$ uv pip install jupyterlab
$.venv/bin/jupyter lab
```
```

=== "Windows"

```
```pwsh-session
PS> uv venv --seed
PS> uv pip install pydantic
PS> uv pip install jupyterlab
PS> .venv\Scripts\jupyter lab
```
```

From here, `import pydantic` will work within the notebook, and you can install additional packages via `!uv pip install`, or even `!pip install`.

Using Jupyter from VS Code

You can also engage with Jupyter notebooks from within an editor like VS Code. To connect a uv-managed project to a Jupyter notebook within VS Code, we recommend creating a kernel for the project, as in the following:

```
# Create a project.
$ uv init project
```

```
# Move into the project directory.
$ cd project

# Add ipykernel as a dev dependency.
$ uv add --dev ipykernel

# Open the project in VS Code.
$ code .
```

Once the project directory is open in VS Code, you can create a new Jupyter notebook by selecting "Create: New Jupyter Notebook" from the command palette. When prompted to select a kernel, choose "Python Environments" and select the virtual environment you created earlier (e.g., `.venv/bin/python` on macOS and Linux, or `.venv\Scripts\python` on Windows).

!!! note

VS Code requires ``ipykernel`` to be present in the project environment. If you'd prefer to avoid adding ``ipykernel`` as a dev dependency, you can install it directly into the project environment with ``uv pip install ipykernel``.

If you need to manipulate the project's environment from within the notebook, you may need to add `uv` as an explicit development dependency:

```
$ uv add --dev uv
```

From there, you can use `!uv add pydantic` to add `pydantic` to the project's dependencies, or `!uv pip install pydantic` to install `pydantic` into the project's virtual environment without updating the project's `pyproject.toml` or `uv.lock` files.